SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	**CSA1205**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING**
Weightage:	**45%**	**Total Marks:**	**25**
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	G. Naveen Kumar Reddy	**Reg.No.:**	192525288
Department:	AIML	**Date:**	

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

4. A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I G . N a v e e n K u m a r R e d d y Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G. Naveen Kumar Reddy

MARKS ALLOCATION

	Problem Understanding (20%)	**Method Selection (20%)**	**Solution Process (25%)**	**Accuracy of Calculation (20%)**	**Interpretation of Results (15%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q1: Arithmetic Operations using 2's Complement

A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation for +45 and −23.

First, convert the numbers into binary.

+45 = 00101101

−23 → 23 = 00010111 → 1's complement = 11101000 → +1 = 11101001

Addition:

00101101 + 11101001 = 00010110 → 22

Subtraction:

45 − (−23) = 45 + 23

00101101 + 00010111 = 01000100 → 68

Overflow occurs only when same sign operands give different sign result. Since signs differ in addition and remain consistent in subtraction, **no overflow occurs**. The processor detects overflow using sign comparison or carry into and out of MSB.

Q2: Carry Look-Ahead Adder (CLA)

A Carry Look-Ahead Adder improves speed by reducing delay compared to ripple carry adders.

Each bit generates:

- Generate (G) = A·B
- Propagate (P) = A ⊕ B

Carries are computed as:

- C1 = G0 + P0C0
- C2 = G1 + P1G0 + P1P0C0
- C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0

Unlike ripple carry adders where carry propagates sequentially (causing delay), CLA computes all carries in parallel.

Comparison:

- Ripple Carry Delay = proportional to number of bits
- CLA Delay = constant (faster)

Thus, CLA improves processor speed by minimizing carry propagation delay.

Q3: Booth's Multiplication Algorithm

Booth's algorithm efficiently multiplies signed numbers like −6 and +5.

Binary:

−6 = 11111010

+5 = 00000101

Registers:

- Accumulator (A) = 00000000
- Multiplier (Q) = 00000101
- Booth bit (Q-1) = 0

Steps depend on Q0 and Q-1:

- 10 → Subtract multiplicand
- 01 → Add multiplicand
- 00 or 11 → No operation

After each step, perform arithmetic right shift.

Booth's algorithm reduces operations by handling consecutive 1's efficiently instead of repeated addition.

Final result:

$-6 \times 5 = -30$

Thus, Booth's algorithm is efficient for signed multiplication.

Q4: Restoring vs Non-Restoring Division

Division example: 13 ÷ 3 → Binary: 1101 ÷ 0011

Restoring Division:

- Subtract divisor from remainder
- If result negative → restore (add back divisor)
- Set quotient bit accordingly

Steps repeat for each bit.

Non-Restoring Division:

- If remainder positive → subtract divisor
- If negative → add divisor in next step
- No restoring step needed

Comparison:

- Restoring: More steps due to restoring operation
- Non-restoring: Faster, fewer operations

Thus, **non-restoring division is preferred** due to efficiency and reduced computation time.

Q5: IEEE 754 Floating Point Representation

Example: −13.25

Convert to binary:

$13.25 = 1101.01 = 1.10101 \times 2^3$

Single Precision (32-bit):

- Sign = 1
- Exponent = 127 + 3 = 130 → 10000010
- Mantissa = 10101000000000000000000

Double Precision (64-bit):

- Sign = 1
- Exponent = 1023 + 3 = 1026
- Mantissa extended to 52 bits

Floating Point Addition:

Steps:

1. Align exponents
2. Add mantissas
3. Normalize result
4. Round if needed

Issues:

- Precision loss
- Rounding errors
- Normalization shifts

Thus, IEEE 754 ensures standardized representation but introduces minor precision limitations.